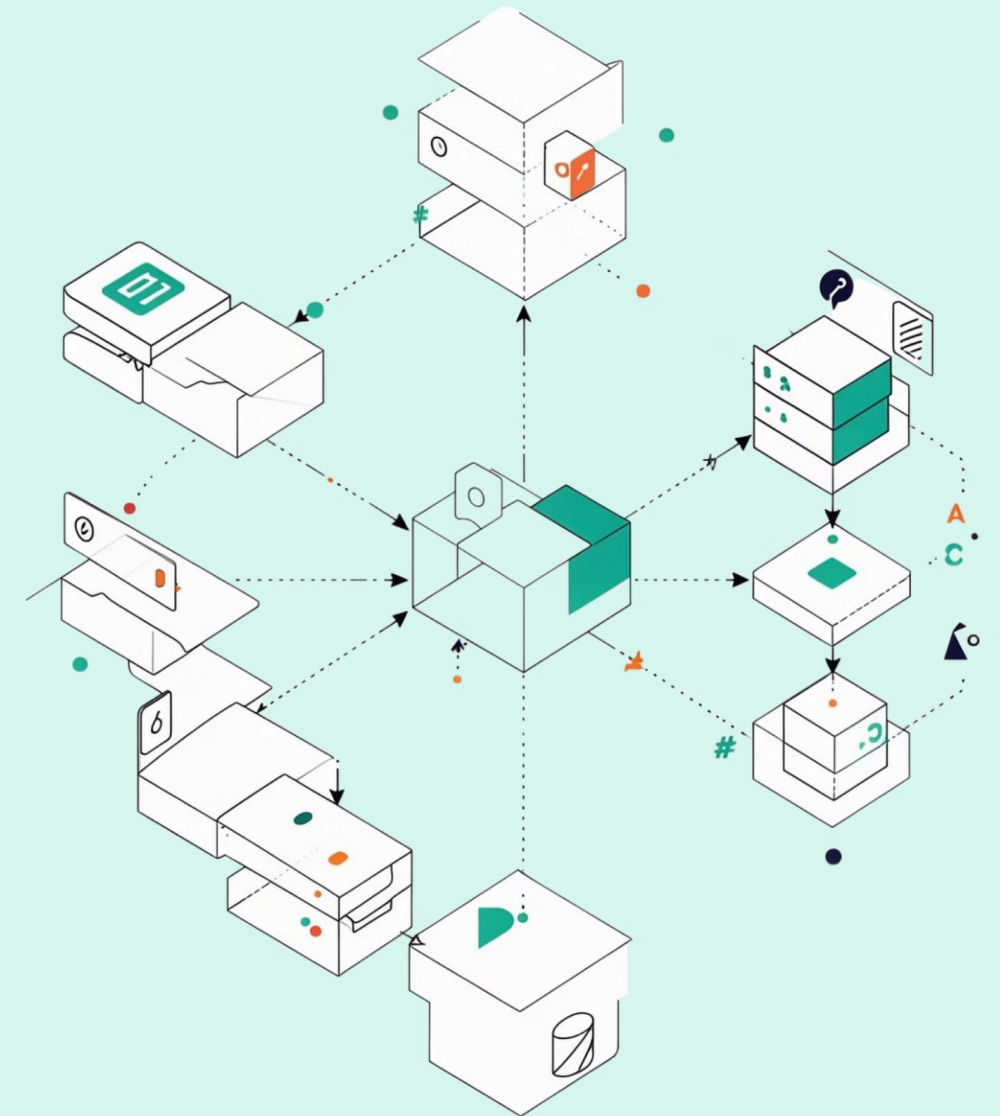


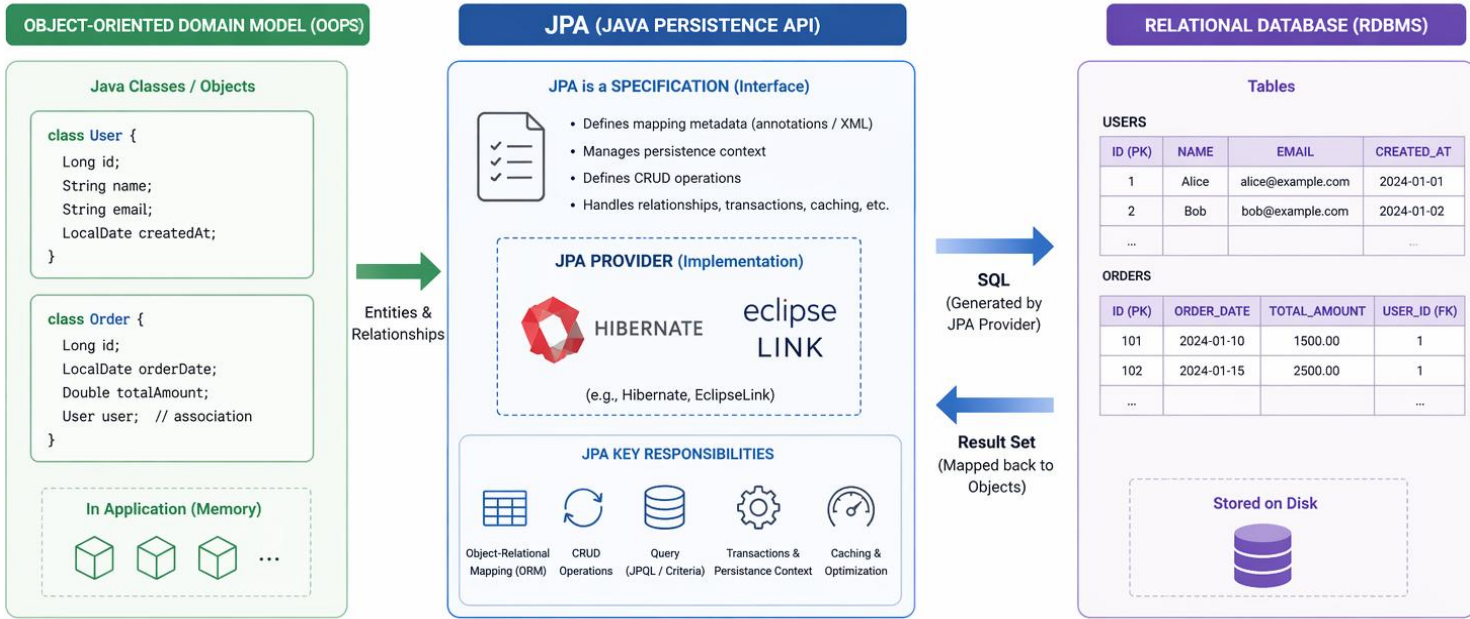
Understanding ORM

1. **Data Persistence**? → storing data permanently (in DB/files) so it survives beyond program execution.
2. **Object-Relational Mapping (ORM)** → programming technique that bridges the gap between object-oriented programming languages and relational databases. It allows developers to work with database data using familiar object-oriented paradigms rather than writing raw SQL queries.
3. **JPA** → Standard Specification for ORM in Java – (Rulebook)
4. **Hibernate** → Implementation of JPA
5. Spring JPA



What is the Java Persistence API (JPA)?

- ❑ The Java Persistence API (JPA) is a Java **specification** that defines how Java objects and classes should be mapped to relational database tables.
- ❑ It serves as a **standardized bridge** between object-oriented domain models and relational database systems, **abstracting** away the complexities of converting data between its object-oriented form in the application and its relational form in the database.



Java Objects

Entity classes with annotations defining database mapping



JPA Specification

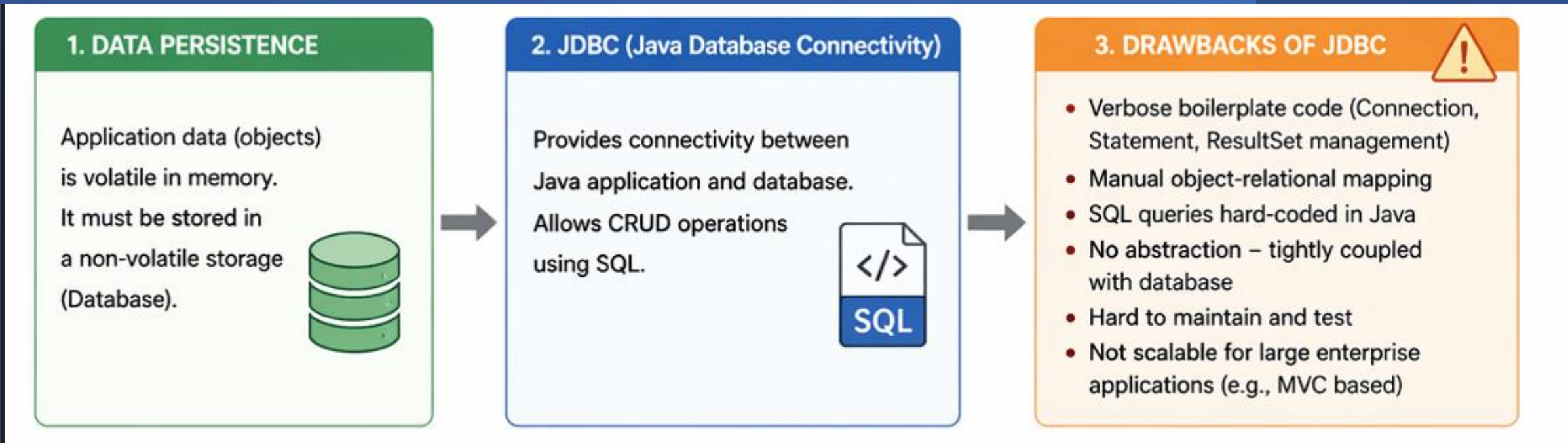
Standardized API for persistence operations and mappings



Relational Database

Tables, columns, and relationships managed by JPA implementation

Popular JPA implementations include Hibernate, EclipseLink, and Apache OpenJPA. This standardization allows developers to switch between implementations with minimal code changes, protecting investments in application architecture.



JDBC (Low-Level)

You control everything (SQL + mapping)
Direct DB interaction
Write SQL manually
Manual mapping
Boilerplate code
Tight coupling with DB
No caching
Hard to scale

Hibernate (High-Level ORM)

Framework controls mapping + SQL for you
Abstraction over DB
Generates SQL automatically
Auto Object ↔ Table mapping
Minimal code
Database independent (mostly)
Built-in caching
Scales better for enterprise apps

4. NEED OF JPA & ORM (SOLUTION)

Why we need JPA & ORM?

- ✓ Eliminate manual mapping
- ✓ Reduce boilerplate code
- ✓ Provide abstraction over JDBC
- ✓ Database independent code
- ✓ Easier maintenance & scalability
- ✓ Focus on business logic, not on data access plumbing

Java Objects (OOP Model)

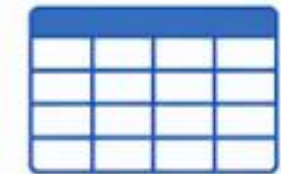


JPA (Java Persistence API)

Standard specification (javax.persistence)
Defines how objects are mapped to database tables
Provides API for CRUD, transactions, queries, relationships, etc.
Abstracts underlying database details
Uses JDBC under the hood



Database Tables (RDBMS)



Hibernate is a Java-based ORM tool that provides a complete framework for mapping application domain objects to relational database tables and vice versa.

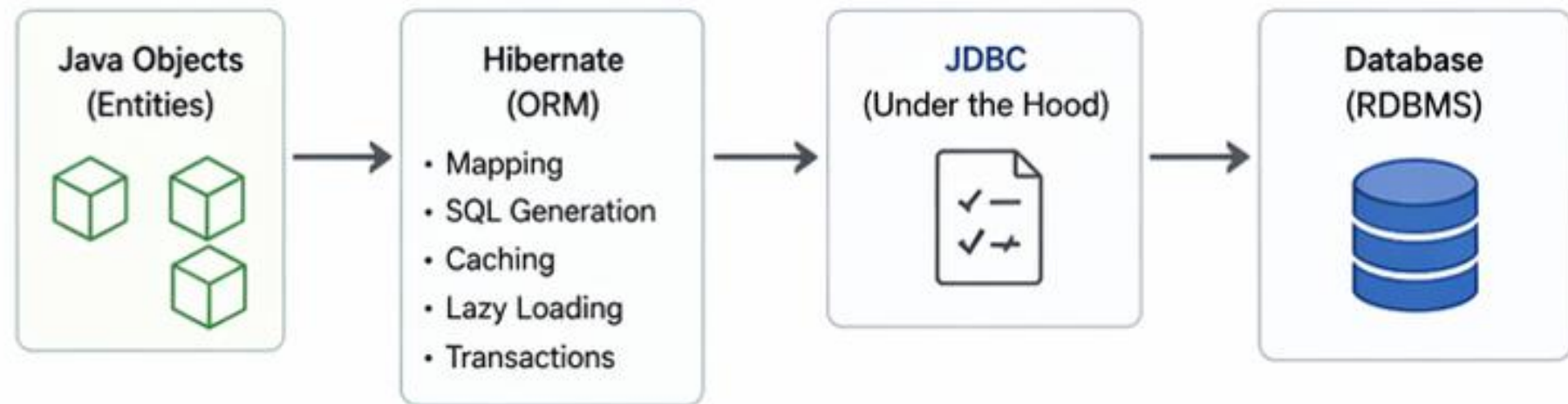
“Spring Data JPA uses JPA, and Hibernate is the most common JPA implementation.”

5. HIBERNATE (ORM IMPLEMENTATION)



- Hibernate is a popular ORM implementation of JPA specification.
- It implements all JPA interfaces and adds additional powerful features.
- Responsible for actual mapping, SQL generation, caching, lazy loading, transactions, etc.
- It still uses JDBC at the bottom to communicate with the database.

How Hibernate Works



The Hibernate Framework

Hibernate sits as an additional layer on top of JDBC, providing a database-independent persistence layer.

It automatically maps database records to Java objects and generates the required SQL statements to replicate all operations to the database. This abstraction dramatically reduces the amount of boilerplate code needed for data persistence while maintaining full control over database operations.



Example: Student Mapping

A `Student` Java class with fields like `id`, `name`, and `email` automatically maps to a `students` database table with corresponding columns. Hibernate handles all the conversion and SQL generation.

Key Features of Hibernate



Transparent Persistence

Hibernate manages the persistence of objects without requiring significant changes to how those objects are designed.



Database Independence

Applications built with Hibernate are portable across different databases with minimal configuration changes.



Performance Optimization

Advanced features like caching mechanisms and lazy loading help optimize performance by reducing unnecessary database access.



Powerful Query Language

Hibernate Query Language (HQL) offers an object-oriented extension to SQL, making data manipulation and retrieval intuitive. Write queries using Java class and property names instead of table and column names.

Batch Processing

Supports batch inserts/updates.
Useful for large data operations.

Automatic Schema Generation

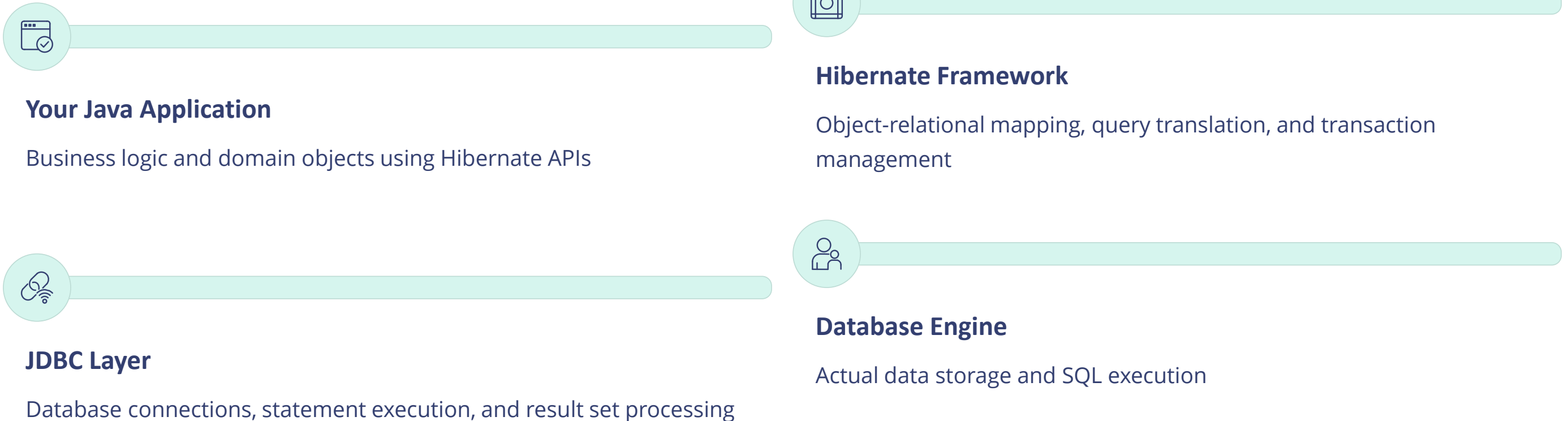
Hibernate can automatically generate database schemas based on your object model

Automatic Table Creation

Can generate/update schema from entity classes (hbm2ddl).
Speeds up development.

Hibernate's Relationship with JDBC

While Hibernate provides a higher-level abstraction, it relies on JDBC for all underlying database communications.



Hibernate acts as an intelligent layer on top of JDBC, handling the complexity of converting between Java objects and database records.

It generates and executes the appropriate JDBC calls based on your object operations, freeing you from writing repetitive data access code while still providing full access to JDBC's capabilities when needed.

Hibernate Architecture Components

1

SessionFactory

A thread-safe, immutable cache of compiled mappings for a single database. This heavyweight object is typically created during application initialization and reused throughout the application lifecycle. It serves as a factory for Session objects and holds configuration information about how Java classes map to database tables.

2

Session

A single-threaded, short-lived object representing a conversation between the application and the database. The Session acts as a factory for Transaction instances and maintains a first-level cache of retrieved data. It's the primary interface for persisting, querying, and retrieving objects.

3

Transaction

A unit of work with the database that represents an abstraction of the application from the underlying transaction implementation. Whether using JTA (Java Transaction API) or JDBC transactions, the Transaction interface provides a consistent way to manage atomic operations.

Benefits of Using Hibernate

Code Efficiency & Flexibility

Significantly reduces boilerplate code associated with JDBC

Advanced Query Capabilities

HQL offers an object-oriented alternative to SQL, seamlessly integrating with Java's object-oriented features like inheritance, polymorphism, and associations. Write queries in terms of your domain model rather than database schema.

Community & Documentation

As an open-source project backed by Red Hat, Hibernate benefits from widespread adoption, extensive documentation, and robust community support.

Integration with Java EE

Seamlessly integrates with Spring Framework and other Java EE technologies

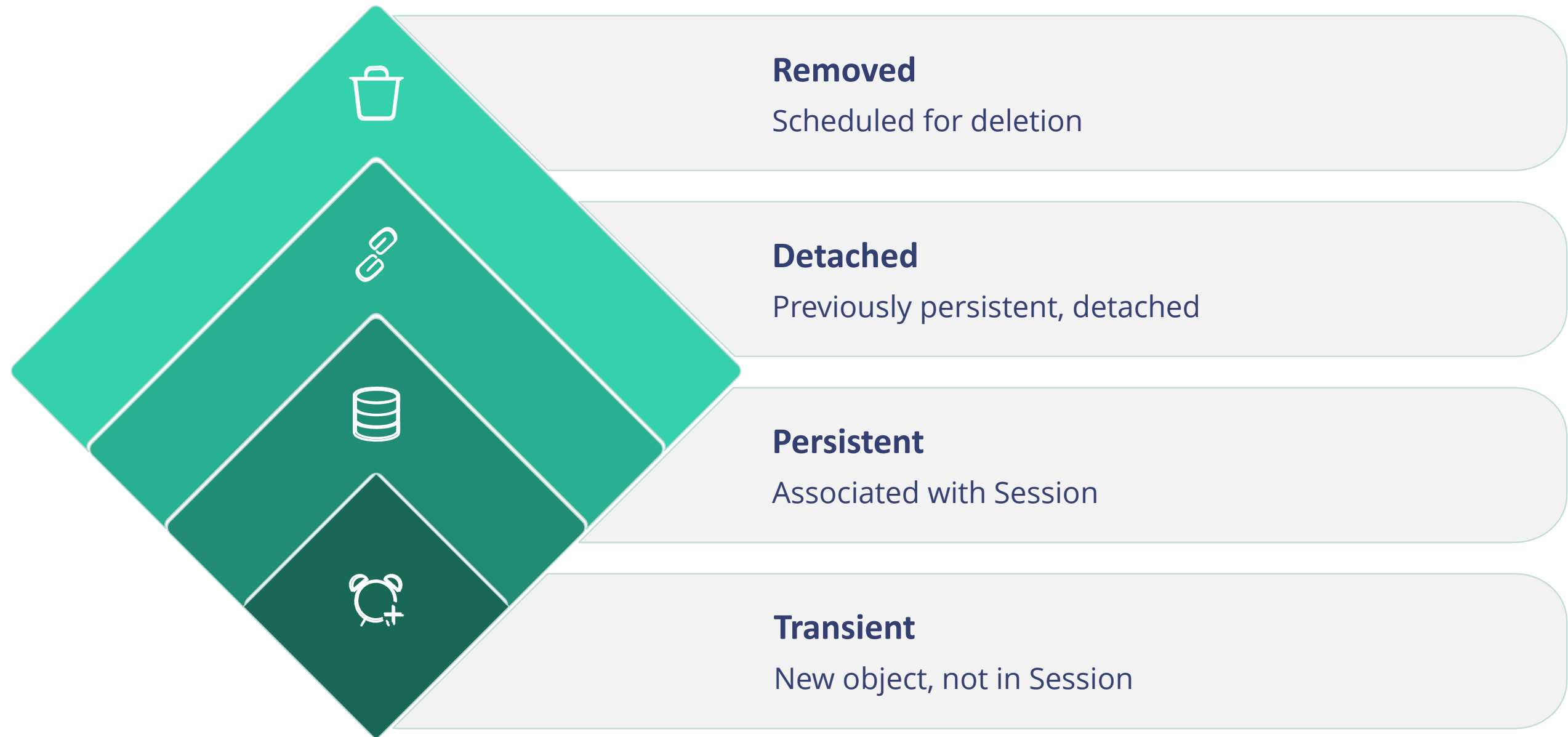
Performance Optimization

Features like lazy loading defer database operations until necessary, while caching mechanisms significantly improve application performance. Multiple levels of caching reduce database round-trips and improve response times.

Vendor-Specific Features

Allows native SQL queries for database-specific optimizations and features when needed.

Entity Lifecycle States in Hibernate



Entity Lifecycle States

How Hibernate tracks your objects — core to understanding unexpected queries

Transient → persist() Persistent → detach() / close() Detached → merge() Persistent → remove() Removed

State transitions in code

// TRANSIENT — not tracked by Hibernate

```
User user = new User("Alice");
```

// PERSISTENT — tracked, changes auto-flushed

```
entityManager.persist(user);
```

```
user.setName("Bob"); // dirty checked → UPDATE issued
```

// DETACHED — no longer tracked

```
entityManager.detach(user);
```

```
user.setName("Charlie"); // no SQL issued
```

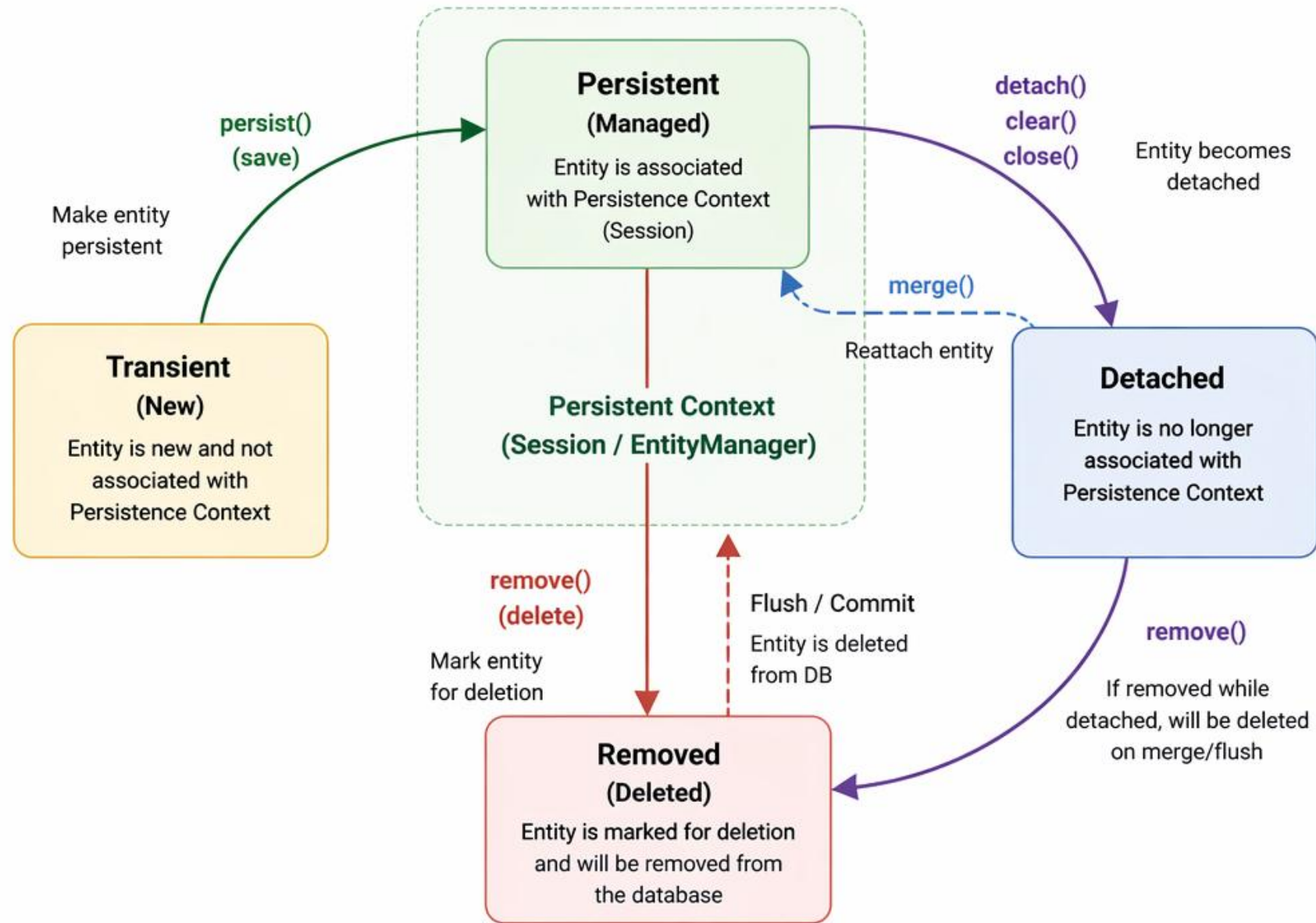
// Back to PERSISTENT via merge()

```
User managed = entityManager.merge(user);
```

// REMOVED — will be deleted on flush

```
entityManager.remove(managed);
```

Hibernate Entity Lifecycle



Annotation	Use
@Entity	Declares a POJO class as a database entity
@Table	Specifies table details (name, schema, constraints)
@Id	Marks primary key field
@GeneratedValue	Auto-generates primary key values
@Column	Maps class field to table column (name, length, unique, nullable)
@Transient	Ignores field (not stored in database)
@Lob	Used for large objects (BLOB/CLOB)
@OrderBy	Sorts data (like SQL ORDER BY)

Design a CRUD-based e-commerce module using Hibernate ORM for the following entities:

- Customer
- Product
- Order
- OrderItem
- Discount

Requirements:

- A Customer can place multiple Orders
- An Order contains multiple OrderItems
- Each OrderItem is linked to one Product
- A Product can have an optional Discount
- Maintain relationships and proper constraints